

1 - Getting Started With Routing

Now that we have Laravel up and running, in this section I want to start off by looking at the file and folder structure. We're also going to talk about the MVC design pattern that Laravel adheres to.

Then we're going to get into routing because that is the entry point of your application. We'll be working in the main routes file and we'll set up endpoints that can be sent an HTTP request and we'll decide how to respond to those requests. So what do we want to do if a get request is made to `/jobs/create` or a post request to `/jobs`. I'll show you how to work with dynamic routes by taking in route parameters for things like IDs and how to add rules and constraints on them. We'll also be looking at the request object and query params and the response helper that laravel provides to work with HTTP requests.

2 - Laravel File & Folder Structure

Laravel has a very well-organized file/folder structure. Let's take a look at the most important folders and files in a Laravel project.

`composer.json`

This file contains all the Composer dependencies for the Laravel application. This includes Laravel itself and any other third-party packages you install. For regular dependencies, we have PHP, Laravel and Laravel Tinker. Tinker is a REPL (Read-Eval-Print Loop) for Laravel. It allows you to interact with your Laravel application from the command line.

Let's look at the dependencies in the `composer.json` file:

Dependencies

- `php` - The PHP version required for the Laravel application.
- `laravel/framework` - The Laravel framework itself.
- `laravel/tinker` - The Laravel Tinker package. This is a REPL (Read-Eval-Print Loop) for Laravel. It allows you to interact with your Laravel application from the command line.

Development Dependencies

- `phpunit/phpunit` - The PHP Unit testing framework.
- `fakerphp/faker` - A PHP library that generates fake data.
- `laravel/pint` - A browser automation and testing tool for Laravel.
- `laravel/sail` - A light-weight command-line interface for interacting with Laravel's default Docker development environment.
- `mockery/mockery` - A simple yet flexible PHP mock object framework for use in unit testing with PHPUnit or PHPSpec.
- `nunomaduro/collision` - An error reporting tool for command-line applications

PSR Autoload Mapping

Let's look at the namespaces being autoloaded using PSR (PHP Standard Recommendation). The PSR-4 standard maps namespaces to directory paths, making it easier to autoload classes without needing to manually include files.

In the Laravel `composer.json` file, you will see a section called `autoload`. This section is used to specify the autoloading rules for the Laravel application.

It looks like this:

```
"autoload": {  
    "psr-4": {  
        "App\\": "app/",  
        "Database\\Factories\\": "database/factories/",  
        "Database\\Seeders\\": "database/seeders/"  
    }  
},
```

What this is doing is mapping the `App` namespace to the `app/` directory, the `Database\\Factories` namespace to the `database/factories/` directory, and the `Database\\Seeders` namespace to the `database/seeders/` directory. This makes it easier to autoload classes without needing to manually include files.

So if we wanted to use a class in the `App` namespace, we would simply use the `App` namespace and the class name. Composer would then autoload the class from the `app/` directory.

package.json

This file contains all the NPM dependencies for the Laravel application. This may seem strange since Laravel is a PHP framework, but often times, Laravel is used as a backend for a JavaScript frontend. This file is used to manage the JavaScript dependencies for the frontend. It includes dependencies like Axios, which is an HTTP client as well as Vite, which is a build tool and frontend development server.

.env

This file contains all the environment variables for the Laravel application. This is where you configure things like the database connection, mail settings, etc. This file is not committed to version control since it contains sensitive information. There is also a `.env.example` file which contains the default values for the environment variables. This one is committed to version control.

vite.config.js

This file contains the configuration for Vite, which is a build tool and frontend development server. This file is used to configure things like the build target, the base URL, etc.

You will also see a file called `artisan`. This is the command-line interface for Laravel. You can use this to run commands like `php artisan serve` to start the development server.

.editorconfig

This file contains the configuration for the editor. This is used to maintain consistent coding styles across different editors and IDEs.

.gitignore

This file contains all the files and folders that should be ignored by Git. This is where you specify the files and folders that should not be committed to version control.

.gitattributes

This file contains the attributes for the files and folders in the Git repository. This is used to specify things like the line endings, the merge strategy, etc.

Let's look at some of the important folders:

- **app** - Contains all the models, controllers, and other PHP classes. Laravel is a Model-View-Controller (MVC) framework. This is where the models and controllers live. Models are used to interact with the database and controllers are used to handle requests and responses. The controllers are in the **app/Http** folder.
- **bootstrap** - Contains the files that bootstrap the Laravel application. This is where the application is initialized.
- **config** - Contains all the configuration files for the Laravel application. You can configure things like the database connection, mail settings, etc.
- **database** - Contains all the database related files. This is where the migrations and seeders live. You also get a default SQLite database file. By default Laravel uses SQLite as the database, but you can easily change it to MySQL, PostgreSQL, or any other database.
- **public** - Contains the public files for the Laravel application. This is where the CSS, JS, and image files live. This is also where the **index.php** file lives which is the entry point for the application.
- **resources** - Contains all the views, language files, and other resources for the Laravel application. This is where the Blade templates live.
- **routes** - Contains all the route files for the Laravel application. This is where you define the routes for your application. This is one of the first places you will look when you are trying to understand how the application works.
- **storage** - Contains all the storage files for the Laravel application. This is where the logs, cache, and other storage files live.
- **tests** - Contains all the test files for the Laravel application. This is where you write your tests.
- **vendor** - Contains all the Composer dependencies for the Laravel application. This is where all the third-party packages live.

3 - MVC: How It Works

We looked at the Laravel folder structure in the last lesson and you saw that there are folders for models, views and controllers, so i just want to take a moment to explain what the Model-View-Controller (MVC) architectural pattern is.

MVC is a design pattern that separates an application into three main components. Each component has a specific role and interacts with the other components in a defined way. I personally think it should have been

called the CMV pattern because the controller is typically the first to receive input from the user, then it processes the input using the model, and finally, it passes the data to the view for display.

A very simple explanation of the MVC pattern is:

- **Controller:** The controller acts as an entry point from the router and an intermediary between the model and the view. It receives input from the user, processes it using the model, and then passes the data to the view for display. In Laravel, controllers are responsible for handling HTTP requests and returning responses.
- **Model:** The model represents the data and business logic of the application. It interacts with the database to retrieve and store data. In Laravel, models are typically used to interact with the database tables.
- **View:** The view is responsible for presenting the data to the user. It is the user interface that the user interacts with. In Laravel, views are typically written in Blade, a templating engine that allows you to write HTML with embedded PHP code.



MVC Psuedo Code:

Let's look at some pseudo code to illustrate how the MVC pattern typically works. This is not actual Laravel code, but it should give you an idea of how the components interact with each other.

```
// Controller
class UserController {
    public function index() {
        $users = User::all();
        return view('users.index', compact('users'));
    }
}

// Model
class User {
    public static function all() {
        return DB::table('users')->get();
    }
}

// View
@foreach ($users as $user)
    <p>{{ $user->name }}</p>
@endforeach
```

In this example, the controller receives a request to display all users. It then uses the User model to retrieve all users from the database. Finally, it passes the users data to the view for display.

So you can see the Controller is the first point of contact and acts as the middleman between the Model and the View. The Model is responsible for interacting with the database and the View is responsible for

presenting the data to the user.

So hopefully this helps a bit. I think understanding the fundamentals of the MVC pattern is important when working with Laravel, as it will help you understand how the different components of the framework fit together.

4 - Intro To Routing

Routing in Laravel is a way to define the endpoints or routes of your application to direct incoming HTTP requests. Meaning that when a user visits a URL, the application should know what to do with it. Usually, a route is connected to some controller method. We'll get into controllers soon, but it doesn't have to load a controller method. You can load a view directly or just return something directly from a function.

Basic Routing

Laravel provides a clean and simple way to define routes. Your route files are located in the `/routes` directory. The main route file is called `web.php`. These files are automatically loaded by Laravel using the configuration in the `bootstrap/app.php` file.

Open the `routes/web.php` file. You should see something like the following:

At the very top of the file, you will see something like this:

```
use Illuminate\Support\Facades\Route;
```

This line is importing the `Route` facade which is used to define routes. A facade is a class that provides a static interface to an underlying class. In this case, the `Route` facade provides a static interface to the `Route` class.

You'll see a lot of imports that gave "illuminate". This is because Laravel's creator, Taylor Otwell, named the framework's core components "Illuminate" to convey the idea of providing clarity and illumination to the process of building web applications.

Below the import statement, you will see a route definition like this:

```
Route::get('/', function () {  
    return view('welcome');  
});
```

Here, we are using the `Route` facade to define a route. The `get` method is used to define a route that responds to HTTP GET requests. The first argument is the URI that the route responds to. In this case, the route responds to the root URI `/`. The second argument is a closure, which is an inline function. In this case, the route function is returning a view names `welcome`. That `welcome` view is located in the `resources/views` directory and is a blade template, which we will also be getting into soon. This is what we see when we visit the root URL of the application.

Returning Values

This particular route is displaying a view, but that isn't the only thing you can do with routes. In many cases, your routes will direct to a controller, which we'll go over in a little bit.

We can just return a string from a route. For example:

```
Route::get('/jobs', function () {  
    return 'Available Jobs';  
});
```

You can also embed HTML in the string:

```
Route::get('/jobs', function () {  
    return '<h1>Available Jobs</h1>';  
});
```

Now when you visit <http://localhost:8000/jobs>, you will see the string `Available Jobs`.

Other HTTP Methods

If you wanted to have a route respond to a POST request, you would use the `post` method. For example:

```
Route::post('/submit', function () {  
    return 'Submitted!';  
});
```

You can test it out using any HTTP client. Here is a CURL request to test it:

```
curl -X POST http://localhost:8000/submit
```

If you get a 419 error, it's because Laravel has built in protection against cross site request forgery (CSRF). Normally you would submit a form to a POST route with a CSRF token. However, if you want to make an exception for specific urls, you can open the `bootstrap/app.php` file and add the following:

```
->withMiddleware(function (Middleware $middleware) {  
    $middleware->validateCsrfTokens(except: [  
        '/submit'  
    ]);  
});
```

Now you can make the request from another domain.

There are all kinds of methods you can use to define routes. Here are a few examples:

```
Route::get($uri, $callback);
Route::post($uri, $callback);
Route::put($uri, $callback);
Route::patch($uri, $callback);
Route::delete($uri, $callback);
Route::options($uri, $callback);
```

Multiple HTTP Verbs

You can also define a route that responds to multiple HTTP verbs. For example:

```
Route::match(['get', 'post'], '/submit', function () {
    return 'Submitted!';
});
```

If you wanted to respond to all HTTP verbs, you could use the `any` method:

```
Route::any('/submit', function () {
    return 'Submitted!';
});
```

Named Routes

You can also give your routes names. This is useful for generating URLs in your views. For example:

```
Route::get('/jobs', function () {
    return 'Available Jobs';
})->name('jobs');
```

Now create a route at `/test` with a link to the `jobs` route:

```
Route::get('/test', function () {
    $url = route('jobs');
    return "<a href='$url'>Click here</a>";
});
```

Now when you visit `http://localhost:8000/test`, you will see a link to the `jobs` route.

Returning JSON

Laravel is also great for building JSON APIs. In Laravel routes you can actually return an array and Laravel will automatically convert it to JSON. For example:

```
Route::get('/api/users', function () {  
    return [  
        'name' => 'John Doe',  
        'email' => 'john@gmail.com',  
    ];  
});
```

Now when you visit <http://localhost:8000/api/users>, you will see the JSON response:

```
{  
  "name": "John Doe",  
  "email": "
```

If you are building an API though, you should consider using Laravel's API routes and Sanctum for authentication. We will go over that in a later chapter.

Redirect Routes

You can also define routes that redirect to another URL. For example:

```
Route::redirect('/here', '/there');
```

Listing Routes

You can also list all the routes in your application by running the following command:

```
php artisan route:list
```

This will show you a list of all the routes in your application, along with the HTTP method, URI, name, and action.

Remove everything except the original code from the `routes/web.php` file. In the next lesson, we will go over route parameters.

5 - Route Paramaters

In the last lesson, we looked at basic routing. In this lesson, we will look at route parameters. Route parameters are used to capture values from the URI. For example, you may have a url like this:


```
http://localhost:8000/user/1
```

In this case, the **1** is a route parameter. This is typically the ID of the resource, in this case, the user.

Open your `routes/web.php` file. You can capture params in your route definition like this:

```
Route::get('/post/{id}', function (string $id) {  
    return 'Post '.$id;  
});
```

In this case, the route parameter is `{id}`. The value of the route parameter is passed to the closure as an argument. In this case, the argument is `$id`. The closure then returns the string `Post` followed by the value of the route parameter.

Now visit `http://localhost:8000/post/1` and you will see the string `Post 1`.

Multiple Route Parameters

You can also have multiple route parameters:

```
Route::get('/posts/{post}/comments/{comment}', function (string $postId, string  
$commentId) {  
    return 'Post '.$postId.' Comment '.$commentId;  
});
```

In this case, we have two route parameters, `{post}` and `{comment}`. The values of these route parameters are passed to the closure as arguments. The closure then returns the string `Post` followed by the value of the `post` route parameter, followed by `Comment` followed by the value of the `comment` route parameter.

Now visit `http://localhost:8000/posts/1/comments/100` and you will see the string `Post 1 Comment 100`.

Optional Route Parameters

You can also have optional route parameters. For example:

```
Route::get('/user/{name?}', function ($name = null) {  
    return $name ? 'User '.$name : 'No user specified';  
});
```

Visit `http://localhost:8000/user` and you will see the string `No user specified`. Visit `http://localhost:8000/user/john` and you will see the string `User john`.

Route Parameter Constraints

You can also constrain route parameters. For example, you may want a route parameter to only accept numbers. You can do this like so:

```
Route::get('/user/{id}', function ($id) {  
    return 'User '.$id;  
})->where('id', '[0-9]+');
```

Now if you visit <http://localhost:8000/user/john>, you will get a 404 error. But if you visit <http://localhost:8000/user/1>, you will see the string `User 1`.

In addition to regular expressions, you can also use the following shortcuts:

- `where('id', '[0-9]+')` is the same as `whereNumber('id')`
- `where('id', '[a-zA-Z]+')` is the same as `whereAlpha('id')`
- `where('id', '[a-zA-Z0-9]+')` is the same as `whereAlphaNumeric('id')`

For example:

```
Route::get('/user/{id}', function ($id) {  
    return 'User '.$id;  
})->whereNumber('id');
```

or

```
Route::get('/user/{id}/{name}', function (string $id, string $name) {  
    return 'User ' . $id . ' ' . $name;  
})->whereNumber('id')->whereAlpha('name');
```

Now if you visit <http://localhost:8000/user/john>, you will get a 404 error. But if you visit <http://localhost:8000/user/1>, you will see the string `User 1`.

Global Constraints

You can also define global constraints in the `App\Providers\AppServiceProvider` class. Open that file and add the following import:

```
use Illuminate\Support\Facades\Route;
```

Then add the following code to the `boot` method:

```
public function boot(): void
{
    Route::pattern('id', '[0-9]+');
}
```

Now use the following route in the `routes/web.php` file:

```
Route::get('/user/{id}', function ($id) {
    return 'User ' . $id;
});
```

Now if you visit `http://localhost:8000/user/john`, you will get a 404 error. But if you visit `http://localhost:8000/user/1`, you will see the string `User 1`. Any route parameter named `id` will now be constrained to only accept numbers.

6 - Request Object & Query Params

In Laravel, you can access the request object in a route closure. The request object contains all the information about the request including the following:

- Request method (GET, POST, PUT, DELETE, etc.)
- Request URI
- Request headers
- Request body
- Query parameters
- Form data
- Uploaded files
- Cookies
- Session data

To access the request object, you can type-hint it in the closure like so:

```
Route::get('/test', function (Illuminate\Http\Request $request) {
    return $request;
});
```

The reason we add the type-hinting is because Laravel automatically includes something called dependency injection when we do so. By specifying `Illuminate\Http\Request $request`, Laravel knows that it needs to pass an instance of `Illuminate\Http\Request` into the method. Without the type hint, Laravel wouldn't know which object to inject. It would be treated as a generic variable.

However, you probably don't want to include the entire namespace every time you want to access the request object. Instead, you can import the `Request` class at the top of the file like so:

```
use Illuminate\Http\Request;
```

Then you can access the request object like this:

```
Route::get('/test', function (Request $request) {  
    return $request;  
});
```

There are many methods to access various parts of the request object. Here are a few examples:

- `$request->method()` - Get the request method (GET, POST, PUT, DELETE, etc.)
- `$request->url()` - Get the request URL
- `$request->header('Content-Type')` - Get a request header
- `$request->all()` - Get all request data (query parameters, form data, etc.)
- `$request->query('name')` - Get a query parameter by name
- `$request->input('name')` - Get a form input by name
- `$request->file('photo')` - Get an uploaded file by name
- `$request->cookie('name')` - Get a cookie by name
- `$request->session()->get('key')` - Get a session value by key

Let's check out some of this stuff in action.

```
Route::get('/test', function (Illuminate\Http\Request $request) {  
    return [  
        'method' => $request->method(),  
        'url' => $request->url(),  
        'path' => $request->path(),  
        'fullUrl' => $request->fullUrl(),  
        'ip' => $request->ip(),  
        'userAgent' => $request->userAgent(),  
        'header' => $request->header(),  
    ];  
});
```

Query Parameters

Here is an example of accessing query parameters:

```
Route::get('/user', function (Request $request) {  
    return $request->query('name');  
});
```

Now visit <http://localhost:8000/user?name=John> and you will see the string `John`.

Multiple Query Parameters

You can also get multiple query parameters like so:

```
Route::get('/user', function (Request $request) {  
    return $request->only(['name', 'age']);  
});
```

Now visit `http://localhost:8000/user?name=John&age=30` and you will see `{"name": "John", "age": "30"}`.

`$request->input`

You can also use `$request->input('name')` to get query parameters. The difference between `query` and `input` is that `query` only gets query parameters, while `input` gets both query parameters and form data.

To get all query parameters, you can use the `all` method:

```
Route::get('/user', function (Request $request) {  
    return $request->all();  
});
```

Check if Query Parameter Exists

You can also check if a query parameter exists like so:

```
Route::get('/user', function (Request $request) {  
    return $request->has('name');  
});
```

Now visit `http://localhost:8000/user?name=John` and you will see `true`. If you visit `http://localhost:8000/user`, you will see `false`.

Default Value

You can also provide a default value if the query parameter doesn't exist:

```
Route::get('/user', function (Request $request) {  
    return $request->input('name', 'Default Name');  
});
```

Now if you visit `http://localhost:8000/user?name=John`, you will see `John`. If you visit `http://localhost:8000/user`, you will see `Default Name`.

You can also exclude query parameters like so:

```
Route::get('/user', function (Request $request) {  
    return $request->except(['name']);  
});
```

Now visit <http://localhost:8000/user?name=John&age=30> and you will see `{"age": "30"}`.

8 - Response Helper

In Laravel, We have a `response()` helper that allows you to return a response object. The response helper can be used for many things such as:

- Setting the HTTP response stats
- Setting Header Values
- Response body, which refers to the main content that the server sends back to the client
- Set Cookies
- Set Session data

Up to this point, we have just been returning strings from our route closures. Laravel does some magic behind the scenes to convert these strings into a response object. However, you can also use the `response()` helper if you want more control over the response.

To return a response object, you can use the `response()` helper function like so:

```
Route::get('/test', function () {  
    return response('Hello, World!');  
});
```

When you call `response()`, it internally references the `Illuminate\Http\Response` class and its methods to construct the appropriate response. It is a global helper function, so you do not need to add any `use` statements to use it.

Status Code

You can set the status code of the response by passing it as the second argument to the `response()` function. For example, to return a **404 Not Found** response, you can do this:

```
Route::get('/test', function () {  
    return response('Hello World', 200);  
});
```

So for a **404 Not Found** response, you can do this:

```
Route::get('/test', function () {  
    return response('Not Found', 404);  
});
```

The `response()` helper is syntactic sugar. Without using the helper, you could do the following:

```
use Illuminate\Http\Response;  
  
Route::get('/test', function () {  
    return new Response('Not Found', 404);  
});
```

Headers

You can set headers on the response by chaining the `header()` method on the response object. For example, to set a `Content-Type` header, you can do this:

```
Route::get('/test', function () {  
    return response('<h1>Hello World</h1>')->header('Content-Type', 'text/plain');  
});
```

Notice, it is not parsing the HTML tags because we set the `Content-Type` header to `text/plain`. If I change it to the following, it will parse the HTML tags:

```
Route::get('/test', function () {  
    return response('<h1>Hello World</h1>')->header('Content-Type', 'text/html');  
});
```

It will also parse if I remove the `Content-Type` header because the default is `text/html`.

Responding with JSON

If we wanted to return a JSON response, Laravel actually does this automatically just by returning an array, but we can do it explicitly:

```
Route::get('/test', function () {  
    return response()->json(['name' => 'John Doe']);  
});
```

Downloading A File

We can download files. The `public_path` helper is used to access the `public` folder, so we could do something like this:

```
Route::get('/download', function () {  
    return response()->download(public_path('favicon.ico'));  
});
```

This will download the favicon in that folder.

Cookies

You can set cookies on the response by chaining the `cookie()` method on the response object. For example, to set a cookie named `name` with the value `John Doe`, you can do this:

```
Route::get('/test', function () {  
    return response('Hello World')->cookie('name', 'John Doe');  
});
```

If you look in the browser's developer tools, you will see the cookie has been set and the value is encrypted. This is for security. However, when you use it, Laravel will decrypt it for you. Let's add a route to decrypt the cookie value:

```
Route::get('/read-cookie', function (Request $request) {  
    $cookieValue = $request->cookie('hello');  
    return response()->json(['cookie' => $cookieValue]);  
});
```

Now visit `http://localhost:8000/read-cookie` and you will see the cookie value.

You can also deal with session data, stream data and much more. I just wanted to give you a taste of what you can do with the response object. You can check out the [official documentation](#) for more information.